Documentação técnica do projeto Challenger 2 – Fiap

Grupo:
Azevedo Renan
Humberto Mauro
Joao Paulo

Arquitetura:

Não adotamos arquitetura alguma e sim o padrão de projeto (design) “MVC”. Como a implementação do MVC é tão central que a arquitetura da aplicação é comumente referida, de forma simplificada, como uma arquitetura “baseada em MVC” com classes concretas.

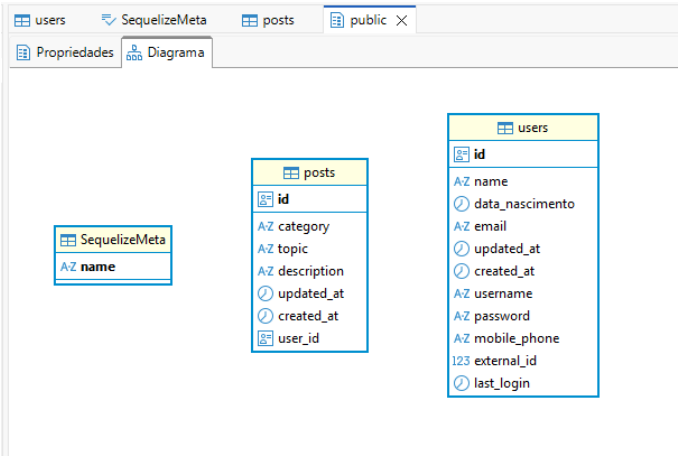
Requisitos técnicos:

Back-end em Node.js, v22.19.0 (mais recente, pelo menos até a escrita desse documento)

Utilização de frameworks Express para roteamento e sequelize (ORM). Migration para criação e alteração de tabelas do banco.

Persistência de dados: Banco de dados Postgressql.

Tabelas do Postgres: criadas e alteradas via ORM sequelizer.
Duas tabelas cradas: posts e users, sem relacionamento de chave estrangeira.
Controle será da aplicação.
A sequelizeMeta é do ORM.



Containerização com Docker: Desenvolvimento e implantação usando contêineres Docker para garantir consistência entre ambientes de desenvolvimento e produção.

Linux: ubuntu latest

Testes unitários: uso do “Axios” para testes.

Foi utilizado o Javascript, nativo do nodes.js

Middleware: Não utilizamos nenhum token.

Setup inicial:

npm init -y
npm install express pg
npm install --save-dev sequelize-cli
npm install axios --save-dev

Execução:
app:

Modo dev: node --watch src/server.js
Modo prod: node src/server.js

sequelize-cli:
criar migration:
npx sequelize-cli migration:generate --name create_users_table

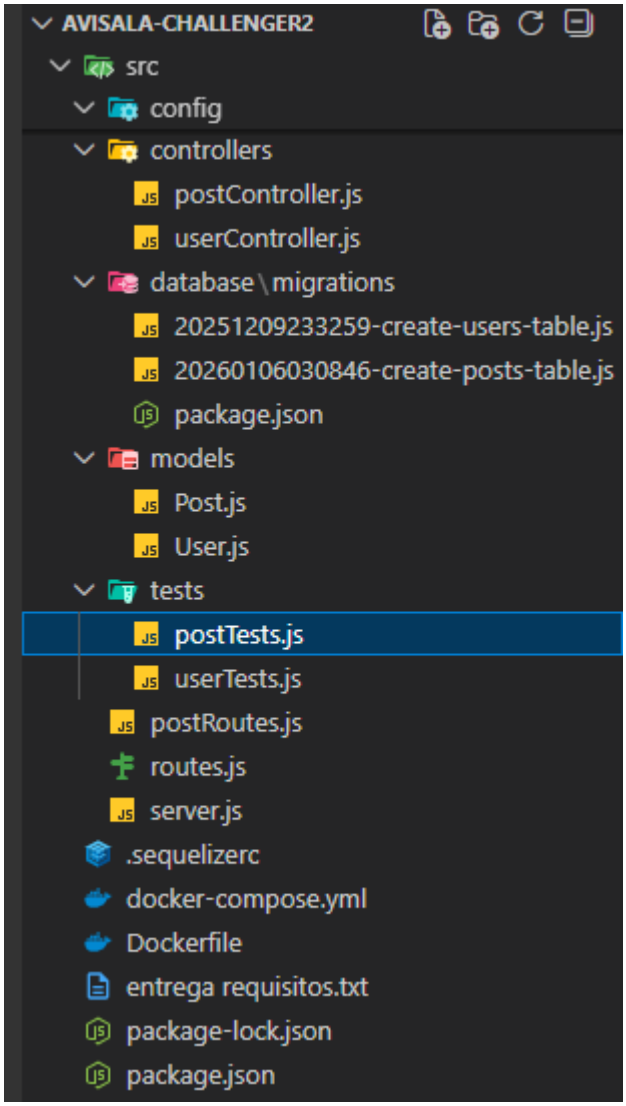
subir (up) migration:
npx sequelize-cli db:migrate

testes:
node src/tests/userTests.js
node src/tests/postTests.js

Editor: VScode e como plug-in o Thunder Client para requisições Restfull

Docker: workflow, Dockerfile e compose.
docker compose up --build

Estrutura:

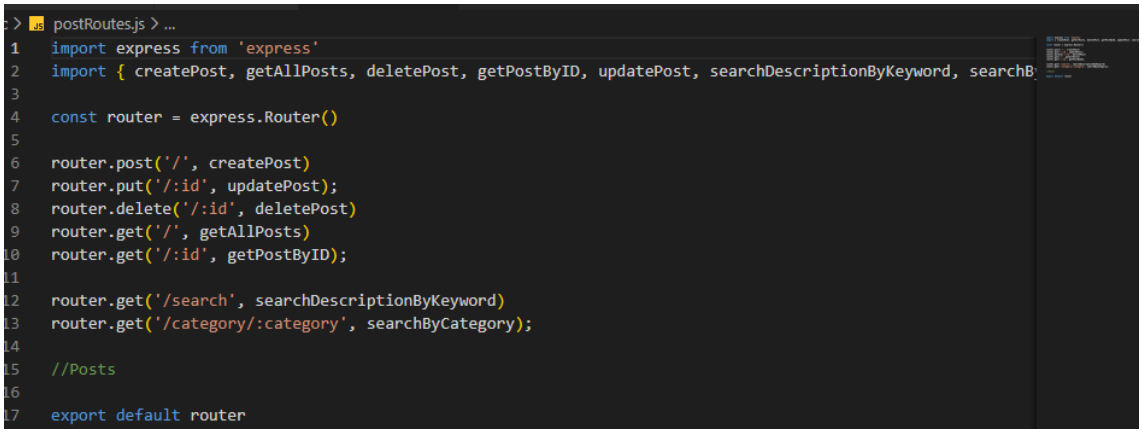


A aplicação se inicia no /src/server.js

Quando um verbo é chamado na URL, a API identifica qual rota chamar pelo que está no server

```
app.use('/users', userRoutes)
app.use('/posts', postRoutes)
```

Direciona para a rota de acordo com que está na url



```
postRoutes.js > ...
import express from 'express'
import { createPost, getAllPosts, deletePost, getPostByID, updatePost, searchDescriptionByKeyword, searchB

const router = express.Router()

router.post('/', createPost)
router.put('/:id', updatePost);
router.delete('/:id', deletePost)
router.get('/', getAllPosts)
router.get('/:id', getPostByID);

router.get('/search', searchDescriptionByKeyword)
router.get('/category/:category', searchByCategory);

//Posts

export default router
```

e executa o método da Controller, de acordo com o parâmetro da rota.

Então temos:

URL://localhost:3000/users

Para users:

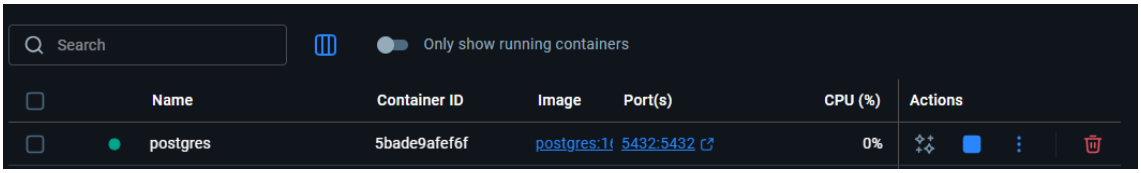
VERBOS	ROTA	AÇÃO
POST	/	createUser
GET	/	getAllUsers
DELETE	/:id	deleteUser
PUT	/:id	updateUser
GET	/:id	getUserByID
GET	/search	searchByPalavraChave
GET	/loginUsuario	loginUsuario

Para Posts:

URL://localhost:3000/posts

VERBOS	ROTA	AÇÃO
POST	/	createPost
GET	/	getAllPosts
DELETE	/:id	deletePost
PUT	/:id	updatePost
GET	/:id	getPostByID
GET	/search	searchDescriptionByKeyword
GET	/category/:category	searchByCategory

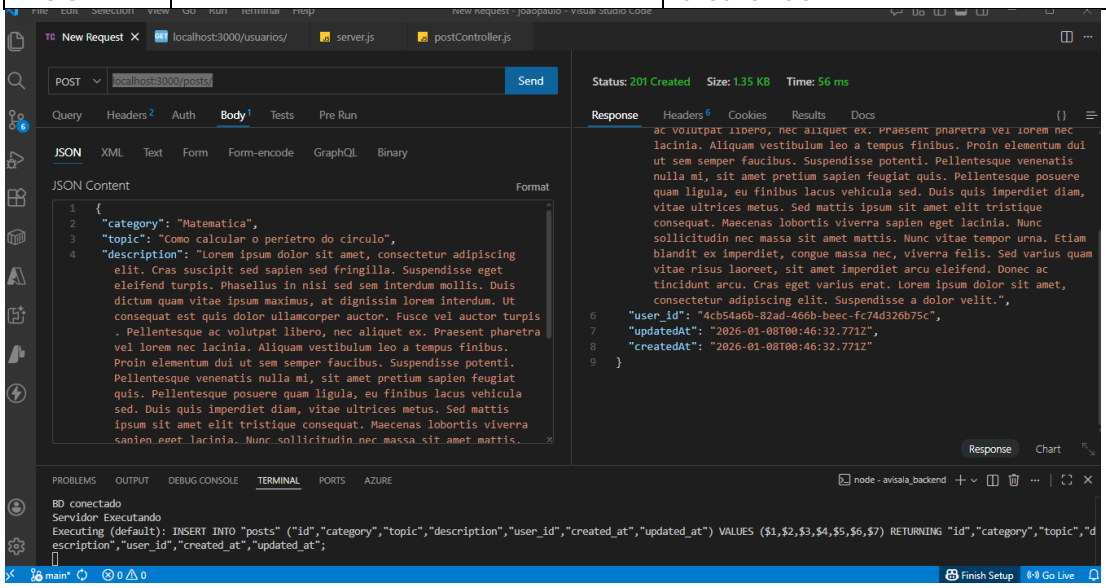
Estando o container do postgres sendo executado e iniciando o app:

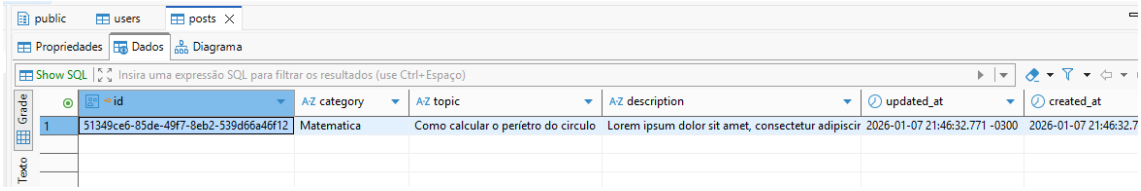


Demonstração:

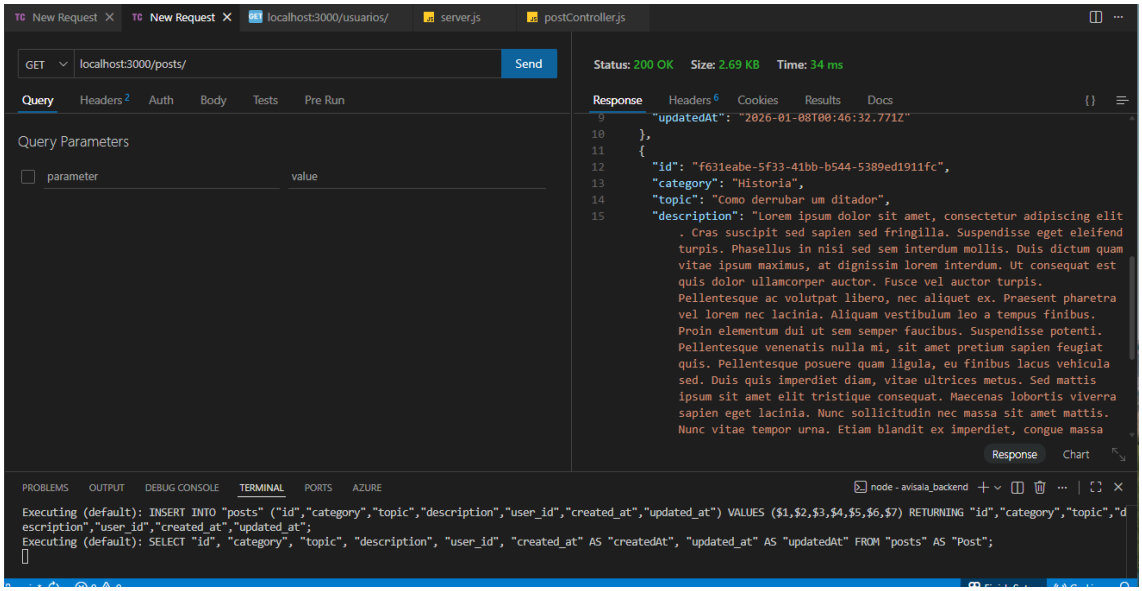
URL: localhost:3000/posts

POST	/	createPost



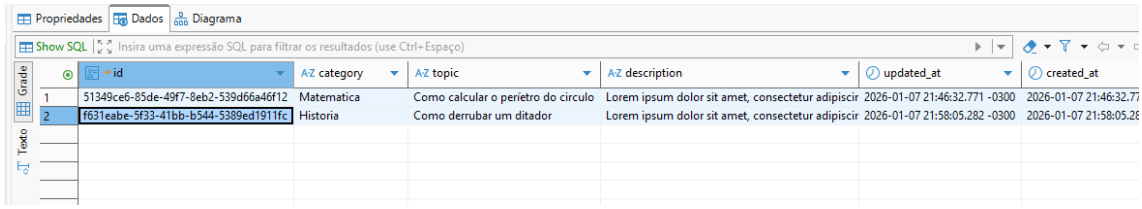


GET	/	getAllPosts



Retorno os 2 posts:
"id": "51349ce6-85de-49f7-8eb2-539d66a46f12", "category": "Matematica", "topic": "Como calcular o perímetro do circulo" ...demais dados e

"id": "f631eabe-5f33-41bb-b544-5389ed1911fc", "category": "Historia", "topic": "Como derrubar um ditador",.. demais dados.



PUT	/:id	updatePost

localhost:3000/posts/f631eabe-5f33-41bb-b544-5389ed1911fc

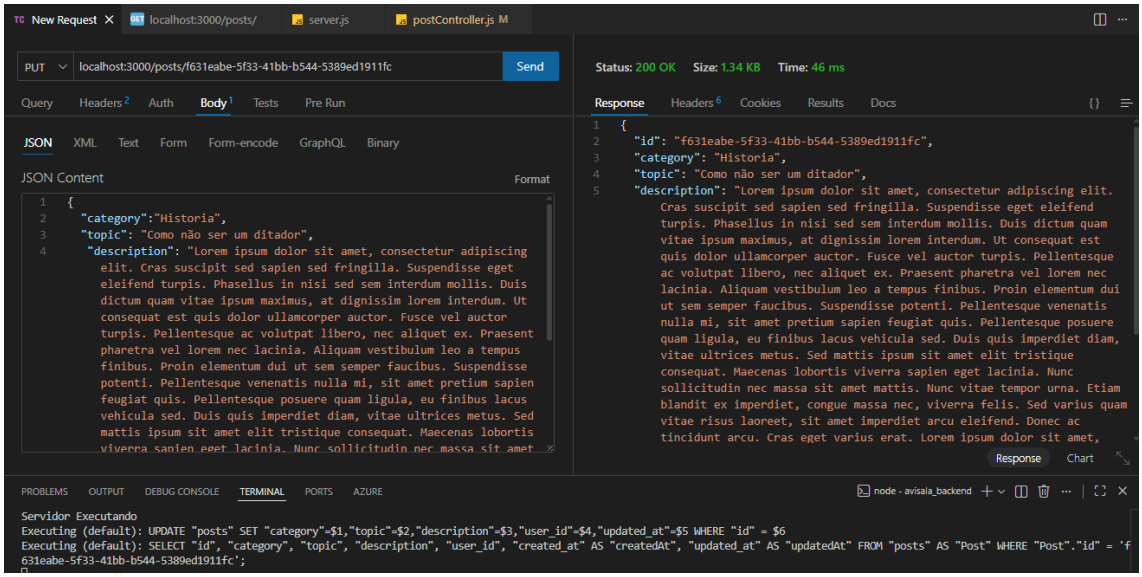
body:

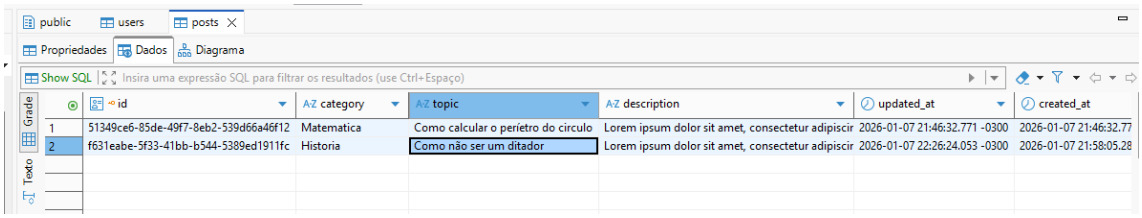
{

"category":"Historia",
"topic": "Como não ser um ditador",
"description": "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras suscipit sed sapien sed fringilla. Suspendisse eget eleifend turpis. Phasellus in nisi sed sem interdum mollis. Duis dictum quam vitae ipsum maximus, at dignissim lorem interdum. Ut consequat est quis dolor ullamcorper auctor. Fusce vel auctor turpis. Pellentesque ac volutpat libero, nec aliquet ex. Praesent pharetra vel lorem nec lacinia. Aliquam vestibulum leo a tempus finibus. Proin elementum dui ut sem semper faucibus. Suspendisse potenti. Pellentesque venenatis nulla mi, sit amet pretium sapien feugiat quis. Pellentesque posuere quam ligula, eu finibus lacus vehicula sed. Duis quis imperdiet diam, vitae ultrices metus. Sed mattis ipsum sit

amet elit tristique consequat. Maecenas lobortis viverra sapien eget lacinia. Nunc sollicitudin nec massa sit amet mattis. Nunc vitae tempor urna. Etiam blandit ex imperdiet, congue massa nec, viverra felis. Sed varius quam vitae risus laoreet, sit amet imperdiet arcu eleifend. Donec ac tincidunt arcu. Cras eget varius erat. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Suspendisse a dolor velit.",
"user_id": "3c3ff0c9-e8d8-441e-ac00-3bbc2ab2a509"

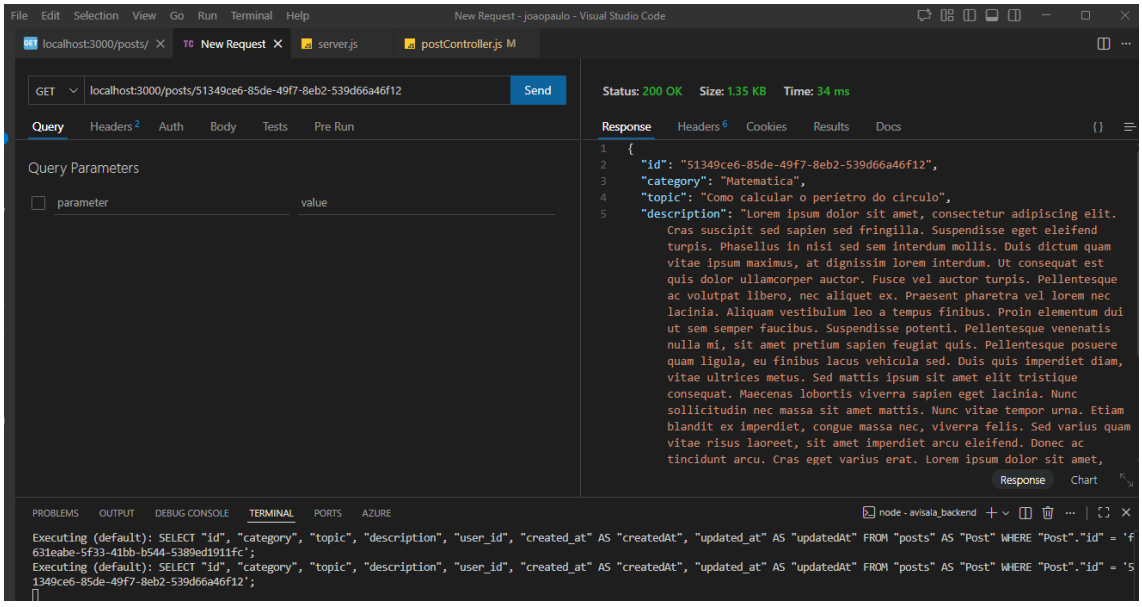
}





GET	/:id	getPostByID

localhost:3000/posts/51349ce6-85de-49f7-8eb2-539d66a46f12



E pode ser também, passando o param “search”, assim passa a atuar como “searchDescriptionByKeyword”

localhost:3000/posts/Search

body

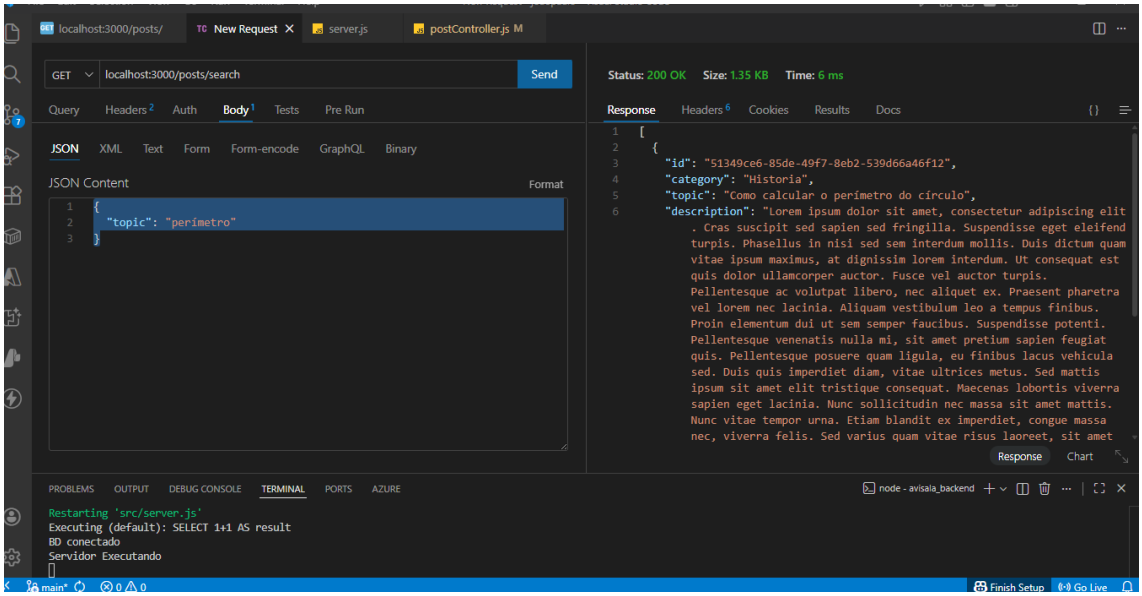
{

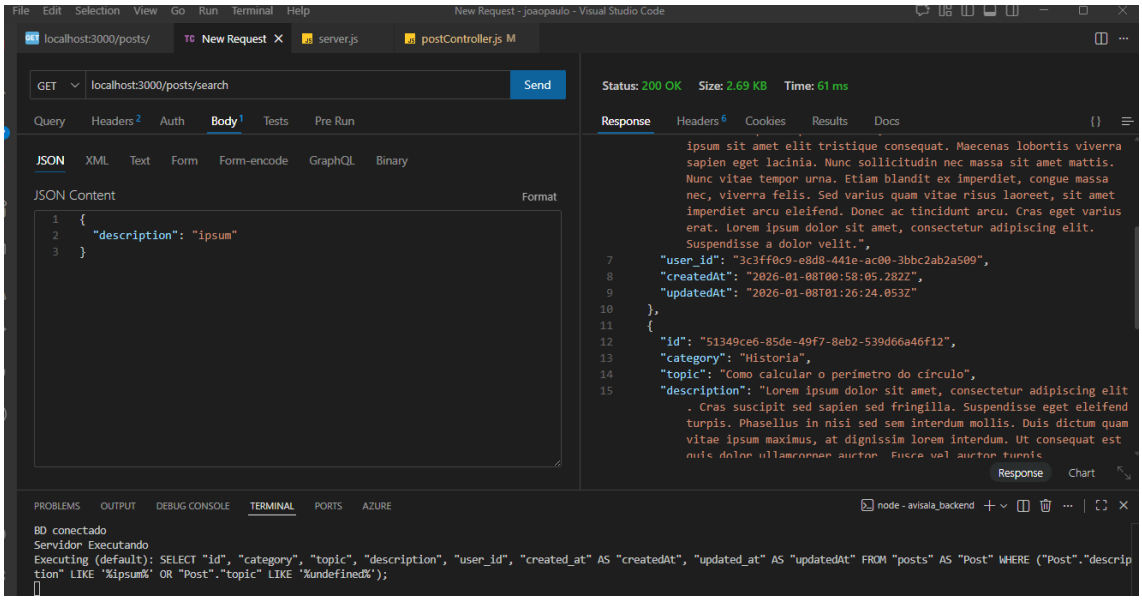
"topic": "perímetro"

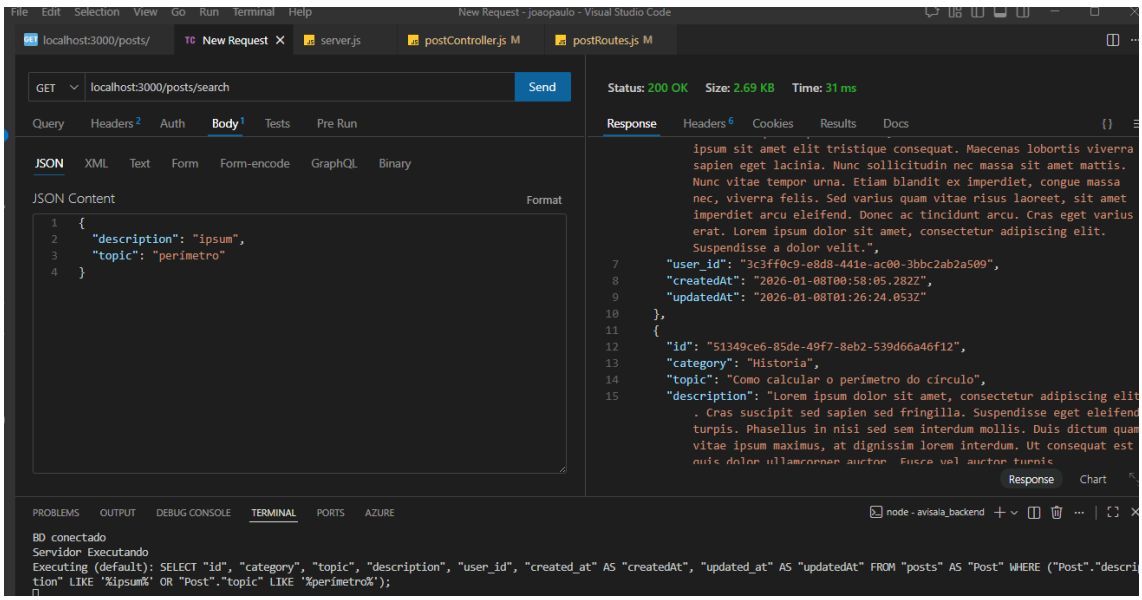
}

A pesquisa pode ser feita pela description ou pelo topic. No caso foi usado o topic.

```
export const searchDescriptionByKeyword = async (req, res) => {
  const posts = await Post.findAll({
    where: {
      [Op.or]: {
        description: {[Op.like]: "%" + req.body.description + "%"},
        topic: {[Op.like]: "%" + req.body.topic + "%"}
      }
    }
  });
  res.status(200).json(posts)
```

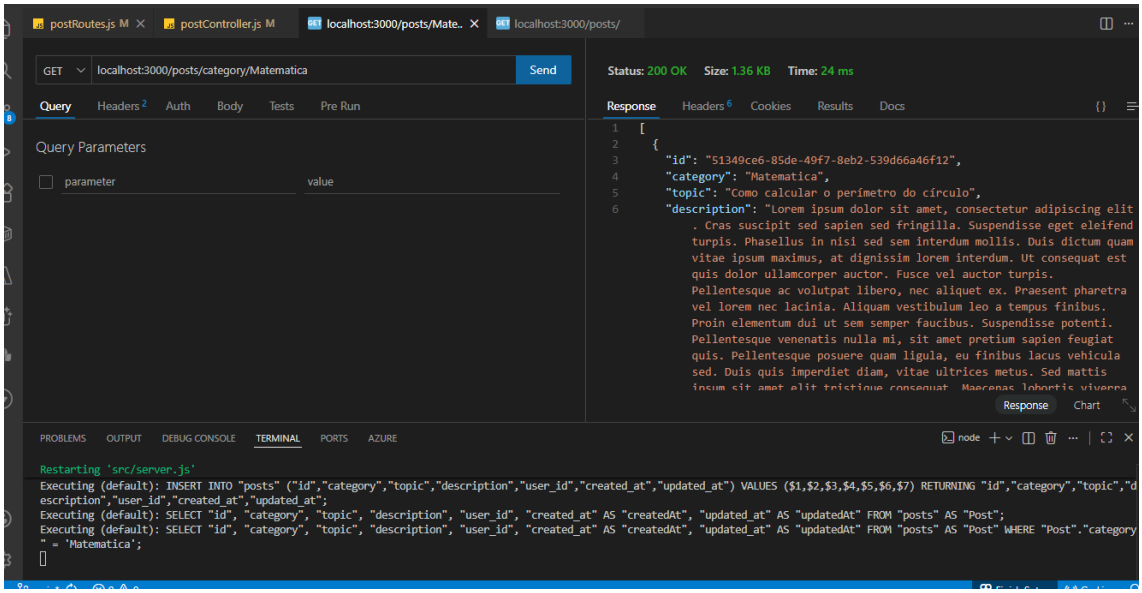

localhost:3000/posts/
New Request
server.js
postController.js M
GET
localhost:3000/posts/search
Send
Status: 200 OK
Size: 1.35 KB
Time: 6 ms
Query
Headers 2
Auth
Body 1
Tests
Pre Run
Response
Headers 6
Cookies
Results
Docs
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
{
"topic": "perímetro"
}
[
{
"id": "51349ce6-85de-49f7-8eb2-539d66a46f12",
"category": "Historia",
"topic": "Como calcular o perímetro do círculo",
"description": "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras suscipit sed sapien sed fringilla. Suspendisse eget eleifend turpis. Phasellus in nisi sed sem interdum mollis. Duis dictum quam vitae ipsum maximus, at dignissim lorem interdum. Ut consequat est quis dolor ullamcorper auctor. Fusce vel auctor turpis. Pellentesque ac volutpat libero, nec aliquet ex. Praesent pharetra vel lorem nec lacinia. Aliquam vestibulum leo a tempus finibus. Proin elementum dui ut sem semper faucibus. Suspendisse potenti. Pellentesque venenatis nulla mi, sit amet pretium sapien feugiat quis. Pellentesque posuere quam ligula, eu finibus lacus vehicula sed. Duis quis imperdiet diam, vitae ultrices metus. Sed mattis ipsum sit amet elit tristique consequat. Maecenas lobortis viverra sapien eget lacinia. Nunc sollicitudin nec massa sit amet mattis. Nunc vitae tempor urna. Etiam blandit ex imperdiet, congue massa nec, viverra felis. Sed varius quam vitae risus laoreet, sit amet
Response
Chart
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
AZURE
node - avisala_backend
Restarting 'src/server.js'
Executing (default): SELECT 1+1 AS result
BD conectado
Servidor Executando
main*
Finish Setup
Go Live


File Edit Selection View Go Run Terminal Help
New Request - joaopaulo - Visual Studio Code
localhost:3000/posts/
New Request
server.js
postController.js M
GET
localhost:3000/posts/search
Send
Status: 200 OK
Size: 2.69 KB
Time: 61 ms
Query
Headers 2
Auth
Body 1
Tests
Pre Run
Response
Headers 6
Cookies
Results
Docs
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
{
"description": "ipsum"
}
ipsum sit amet elit tristique consequat. Maecenas lobortis viverra sapien eget lacinia. Nunc sollicitudin nec massa sit amet mattis. Nunc vitae tempor urna. Etiam blandit ex imperdiet, congue massa nec, viverra felis. Sed varius quam vitae risus laoreet, sit amet imperdiet arcu eleifend. Donec ac tincidunt arcu. Cras eget varius erat. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Suspendisse a dolor velit.",
"user_id": "3c3ff0c9-e8d8-441e-ac00-3bbc2ab2a509",
"createdAt": "2026-01-08T00:58:05.282Z",
"updatedAt": "2026-01-08T01:26:24.053Z"
},
{
"id": "51349ce6-85de-49f7-8eb2-539d66a46f12",
"category": "Historia",
"topic": "Como calcular o perímetro do círculo",
"description": "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras suscipit sed sapien sed fringilla. Suspendisse eget eleifend turpis. Phasellus in nisi sed sem interdum mollis. Duis dictum quam vitae ipsum maximus, at dignissim lorem interdum. Ut consequat est
Response
Chart
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
AZURE
node - avisala_backend
BD conectado
Servidor Executando
Executing (default): SELECT "id", "category", "topic", "description", "user_id", "created_at" AS "createdAt", "updated_at" AS "updatedAt" FROM "posts" AS "Post" WHERE ("Post"."description" LIKE '%ipsum%' OR "Post"."topic" LIKE '%undefined%');


File Edit Selection View Go Run Terminal Help
New Request - joaopaulo - Visual Studio Code
localhost:3000/posts/
New Request
server.js
postController.js M
postRoutes.js M
GET
localhost:3000/posts/search
Send
Status: 200 OK
Size: 2.69 KB
Time: 31 ms
Query
Headers 2
Auth
Body 1
Tests
Pre Run
Response
Headers 6
Cookies
Results
Docs
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
{
"description": "ipsum",
"topic": "perímetro"
}
ipsum sit amet elit tristique consequat. Maecenas lobortis viverra sapien eget lacinia. Nunc sollicitudin nec massa sit amet mattis. Nunc vitae tempor urna. Etiam blandit ex imperdiet, congue massa nec, viverra felis. Sed varius quam vitae risus laoreet, sit amet imperdiet arcu eleifend. Donec ac tincidunt arcu. Cras eget varius erat. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Suspendisse a dolor velit.",
"user_id": "3c3ff0c9-e8d8-441e-ac00-3bbc2ab2a509",
"createdAt": "2026-01-08T00:58:05.282Z",
"updatedAt": "2026-01-08T01:26:24.053Z"
},
{
"id": "51349ce6-85de-49f7-8eb2-539d66a46f12",
"category": "Historia",
"topic": "Como calcular o perímetro do círculo",
"description": "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras suscipit sed sapien sed fringilla. Suspendisse eget eleifend turpis. Phasellus in nisi sed sem interdum mollis. Duis dictum quam vitae ipsum maximus, at dignissim lorem interdum. Ut consequat est
Response
Chart
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
AZURE
node - avisala_backend
BD conectado
Servidor Executando
Executing (default): SELECT "id", "category", "topic", "description", "user_id", "created_at" AS "createdAt", "updated_at" AS "updatedAt" FROM "posts" AS "Post" WHERE ("Post"."description" LIKE '%ipsum%' OR "Post"."topic" LIKE '%perímetro%');

GET	/:category	searchByCategory

localhost:3000/posts/Matematica

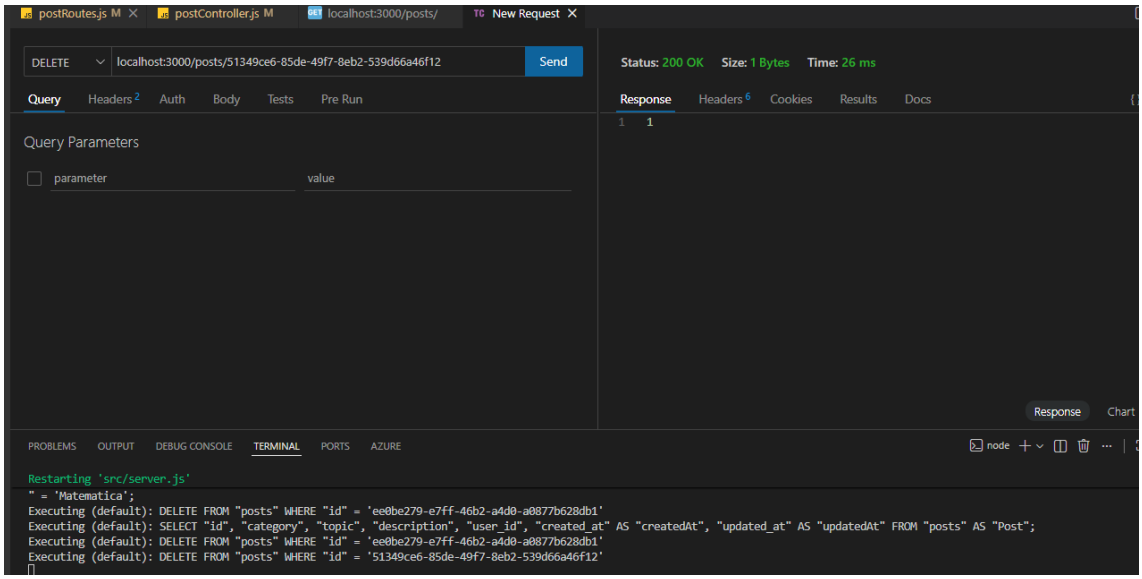


Só há 1 registro com categoria Matematica

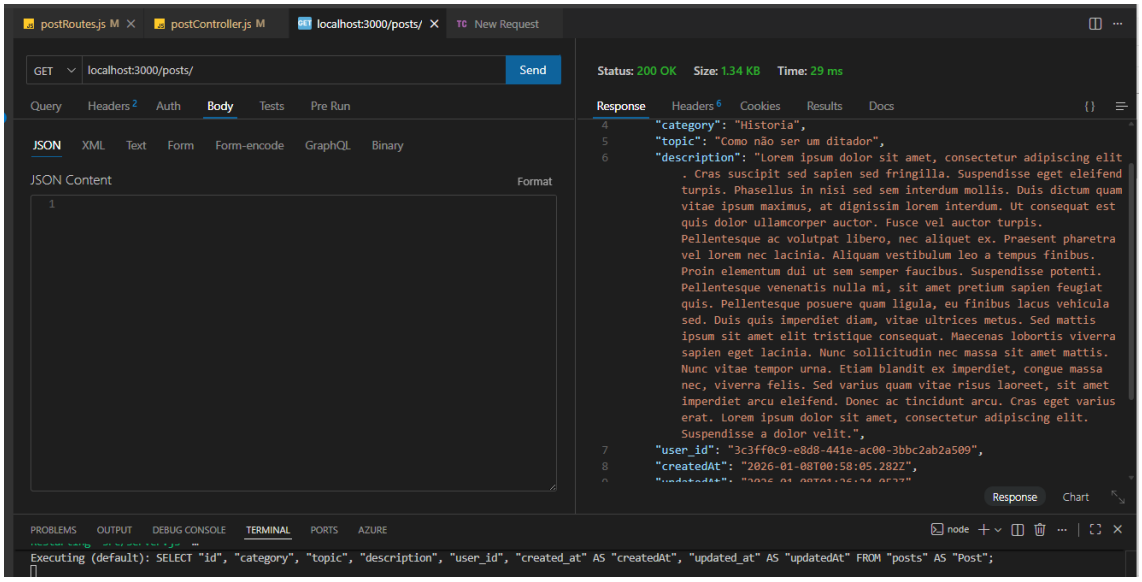
DELETE	/:id	deletePost

localhost:3000/posts/51349ce6-85de-49f7-8eb2-539d66a46f12

Executing (default): DELETE FROM "posts" WHERE "id" = '51349ce6-85de-49f7-8eb2-539d66a46f12'

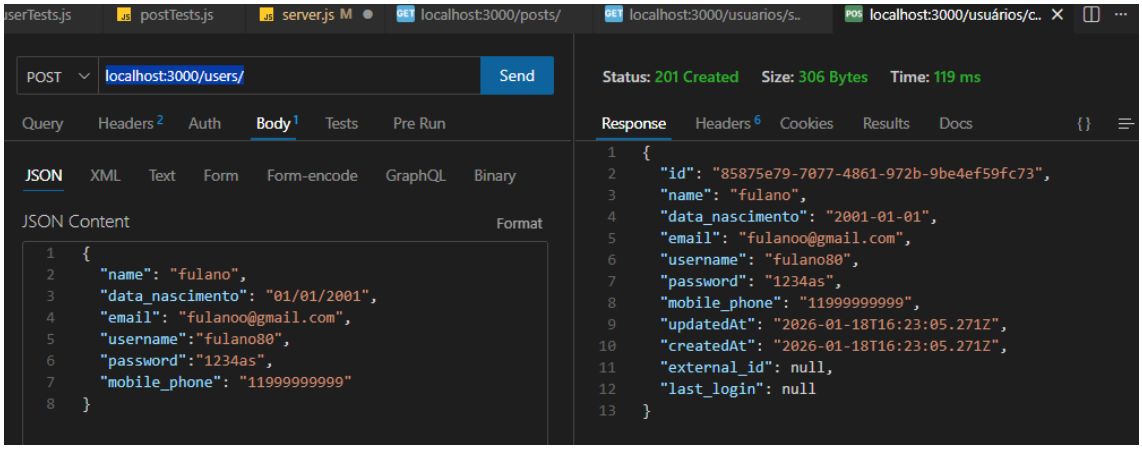


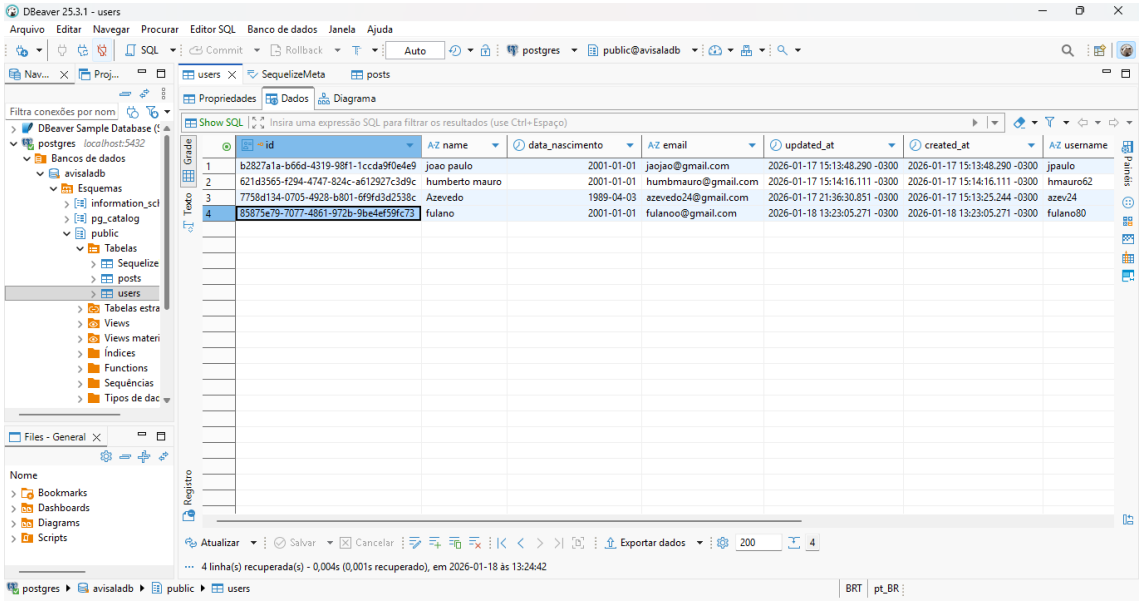
E por fim, listando todos os posts


postRoutes.js M
postController.js M
localhost:3000/posts/
New Request
GET
localhost:3000/posts/
Send
Query
Headers 2
Auth
Body
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
Status: 200 OK
Size: 1.34 KB
Time: 29 ms
Response
Headers 6
Cookies
Results
Docs
"category": "Historia",
"topic": "Como não ser um ditador",
"description": "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras suscipit sed sapien sed fringilla. Suspendisse eget eleifend turpis. Phasellus in nisi sed sem interdum mollis. Duis dictum quam vitae ipsum maximus, at dignissim lorem interdum. Ut consequat est quis dolor ullamcorper auctor. Fusce vel auctor turpis. Pellentesque ac volutpat libero, nec aliquet ex. Praesent pharetra vel lorem nec lacinia. Aliquam vestibulum leo a tempus finibus. Proin elementum dui ut sem semper faucibus. Suspendisse potenti. Pellentesque venenatis nulla mi, sit amet pretium sapien feugiat quis. Pellentesque posuere quam ligula, eu finibus lacus vehicula sed. Duis quis imperdiet diam, vitae ultrices metus. Sed mattis ipsum sit amet elit tristique consequat. Maecenas lobortis viverra sapien eget lacinia. Nunc sollicitudin nec massa sit amet mattis. Nunc vitae tempor urna. Etiam blandit ex imperdiet, congue massa nec, viverra felis. Sed varius quam vitae risus laoreet, sit amet imperdiet arcu eleifend. Donec ac tincidunt arcu. Cras eget varius erat. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Suspendisse a dolor velit.",
"user_id": "3c3ff0c9-e8d8-441e-ac00-3bbc2ab2a509",
"createdAt": "2026-01-08T00:58:05.282Z",
Response
Chart
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
AZURE
node
Executing (default): SELECT "id", "category", "topic", "description", "user_id", "created_at" AS "createdAt", "updated_at" AS "updatedAt" FROM "posts" AS "Post";

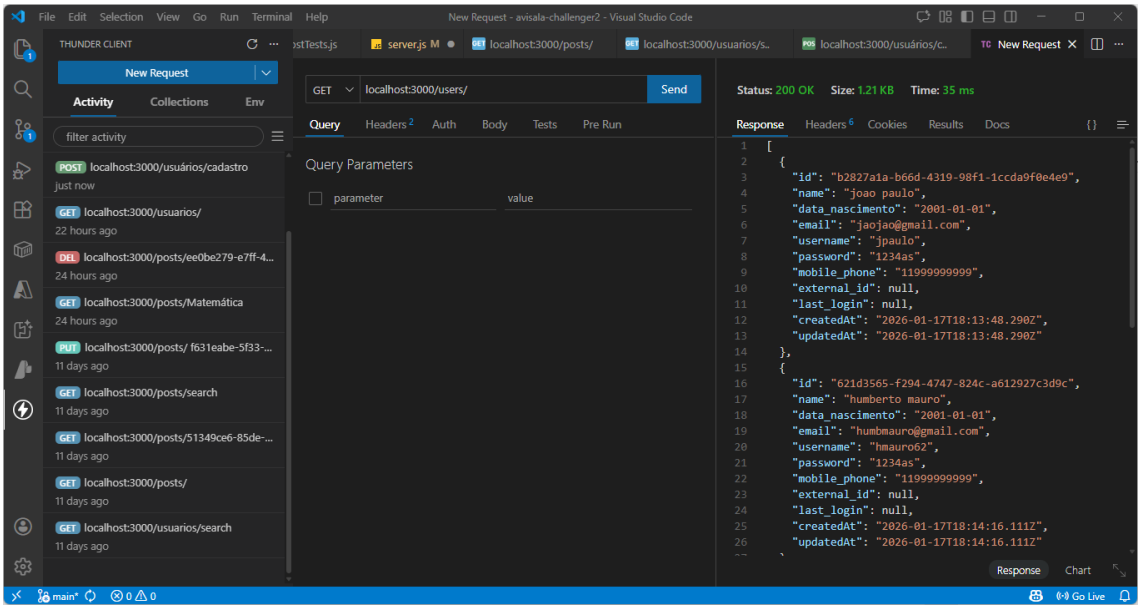
POST	/	createUser

url: localhost:3000/users/



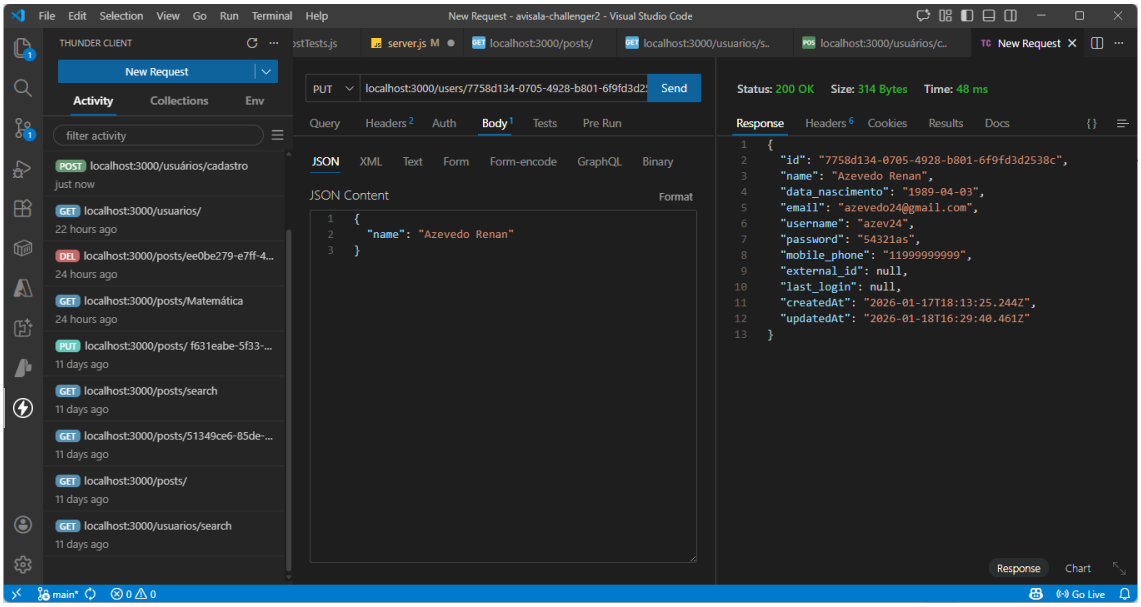


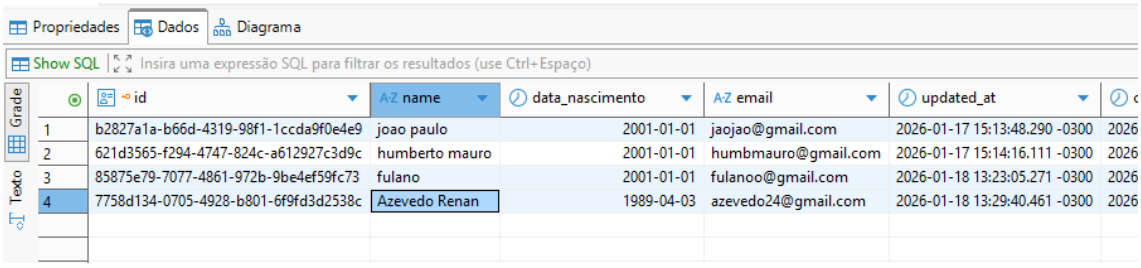
GET	/	getAllUsers



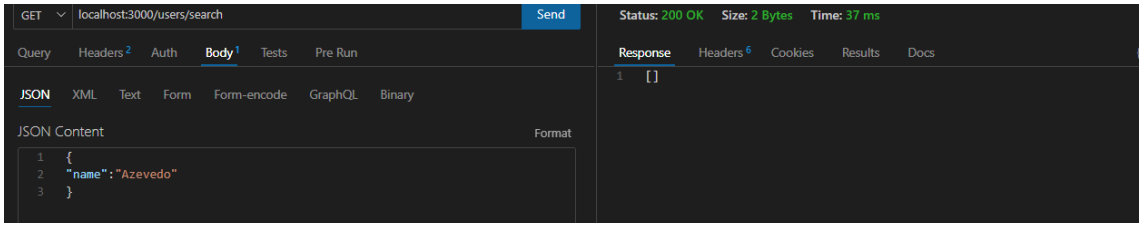
PUT	/:id	updateUser

localhost:3000/users/7758d134-0705-4928-b801-6f9fd3d2538c

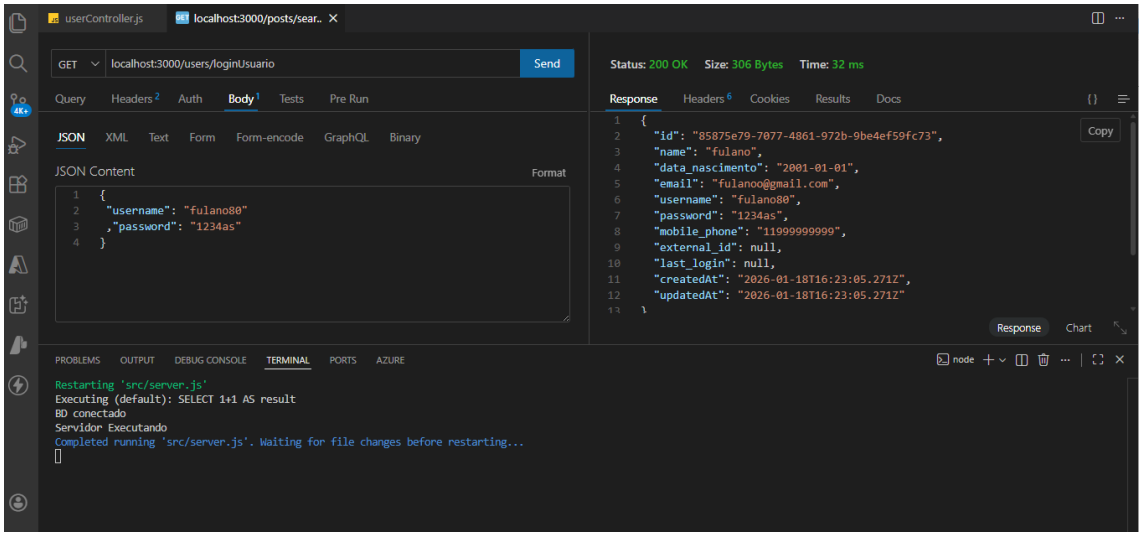




GET	/search	searchByPalavraChave



GET	/loginUsuario	loginUsuario



Relatos de experiências e desafios enfrentados:

João Paulo:

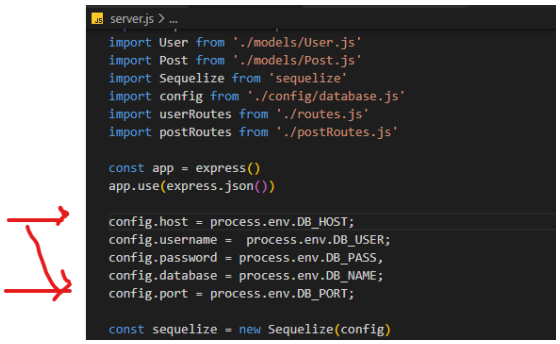
Quando executou o comando “docker compose up –build” o container do app enxergava o container do postgres, porém haviam erros na execução.

Após buscas no google, foi identificado que a versão do postgres deveria ser a 16 e não a 18. (identificado pelo Azevedo)

no docker-composer.yml: alterada a linha da imagem:

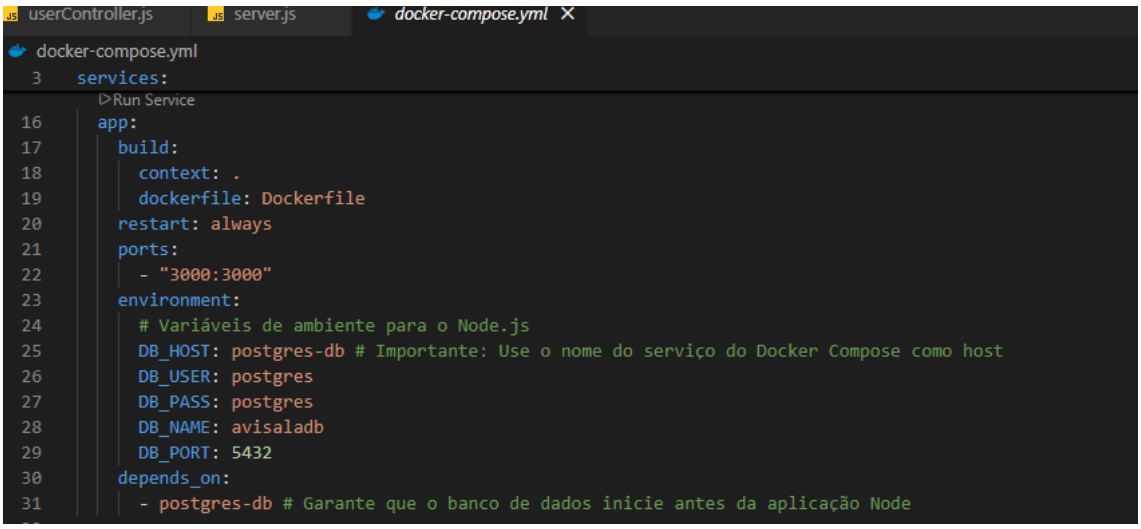
image: postgres:16-alpine # Usar uma imagem oficial e leve do Postgres

Foi necessário também, criar no server.js as instruções de config e subir novamente para o container:



```
server.js > ...
import User from './models/User.js'
import Post from './models/Post.js'
import Sequelize from 'sequelize'
import config from './config/database.js'
import userRoutes from './routes.js'
import postRoutes from './postRoutes.js'

const app = express()
app.use(express.json())

config.host = process.env.DB_HOST;
config.username =  process.env.DB_USER;
config.password = process.env.DB_PASS,
config.database = process.env.DB_NAME;
config.port = process.env.DB_PORT;

const sequelize = new Sequelize(config)
```

E no docker-compose.yml



```
userController.js   server.js   docker-compose.yml
docker-compose.yml
3   services:
      Run Service
16    app:
17      build:
18        context: .
19        dockerfile: Dockerfile
20      restart: always
21      ports:
22        - "3000:3000"
23      environment:
24        # Variáveis de ambiente para o Node.js
25        DB_HOST: postgres-db # Importante: Use o nome do serviço do Docker Compose como host
26        DB_USER: postgres
27        DB_PASS: postgres
28        DB_NAME: avisaladb
29        DB_PORT: 5432
30      depends_on:
31        - postgres-db # Garante que o banco de dados inicie antes da aplicação Node
```

Azevedo: criou o workflow, o Dockerfile e docker-compose, pesquisando no google e nos vídeos do curso.
Os erros encontrados foram trabalhados em conjunto com o João Paulo. O primeiro a criar a imagem no github.

Humberto: Remontei o api em minha máquina, clonando o git do João e transferir para o meu.

A primeira tentativa de subir o container, deu erro:

ERROR: failed to build: **invalid tag "***/avisala_backend-node-app:latest":** invalid reference format

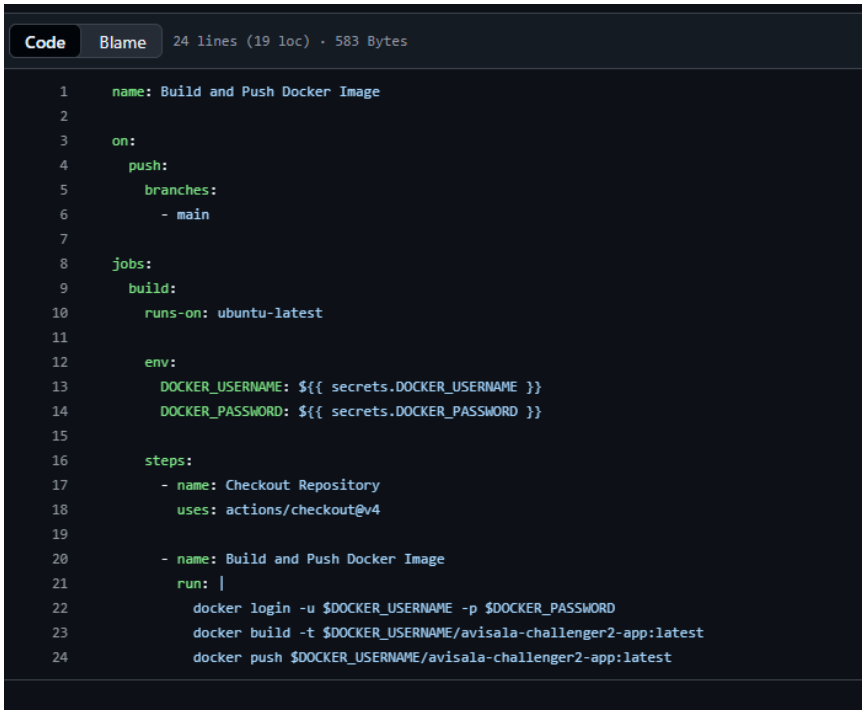
Como o dockerfile não tinha erro algum, resolvi alterar a tag da imagem para “avisala-challenger2”, por ser mais curta.

docker tag 63dcac607943 'avisala-challenger2'

e deletei as anteriores, com o container *stopado*.

docker rmi avisala-challenger2-node-app:latest

Alterei o workflow:



E refiz a subida.

Passou a dar outro erro:

ERROR: docker: 'docker buildx build' requires 1 argument

Usage: docker buildx build [OPTIONS] PATH | URL | -

Run 'docker buildx build --help' for more information

Error: Process completed with exit code 1.

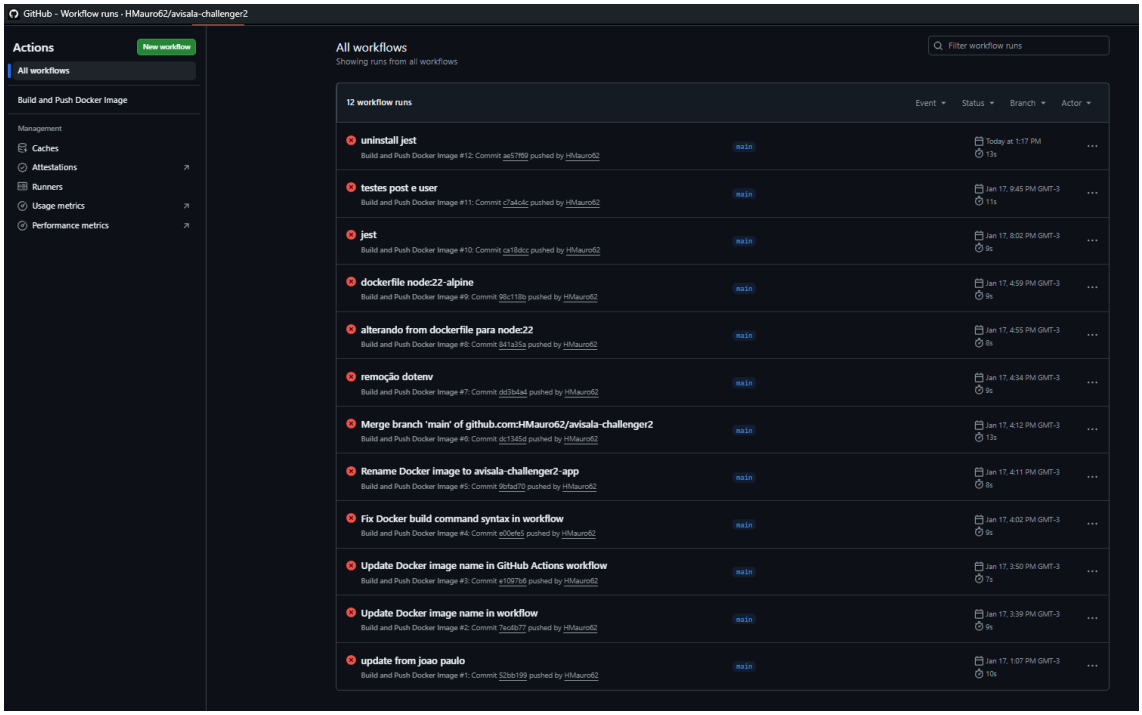
A pesquisa que realizei indica que ou o Dockerfile está com erro ou o buildx está corrompido.
O buildx faz parte do Docker. Então desinstalei o Docker desktop, baixei um novo e instalei novamente. Loguei e refiz o processo. Mesmo erro.

Cheguei a alterar a versão do nodes pois da 20 a 22 há muitas melhorias, mas também não deu certo.

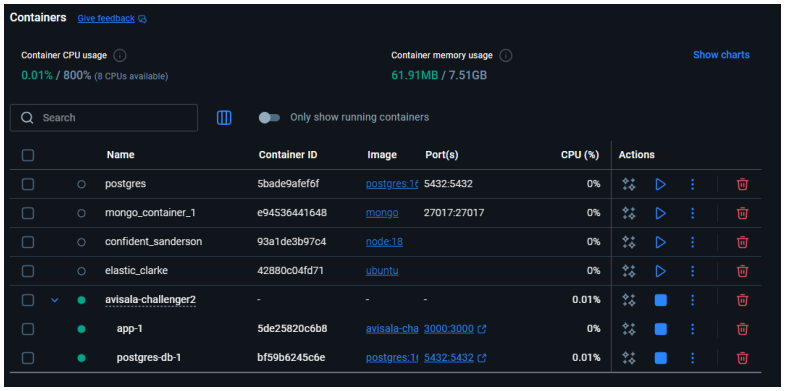
```dockerfile
FROM node:22-alpine

WORKDIR /usr/src/app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["node", "src/server.js"]
```

Não dei continuidade. Não consegui fazer o CI e nem o CD.

Essas são minhas Actions:



No meu Docker desktop

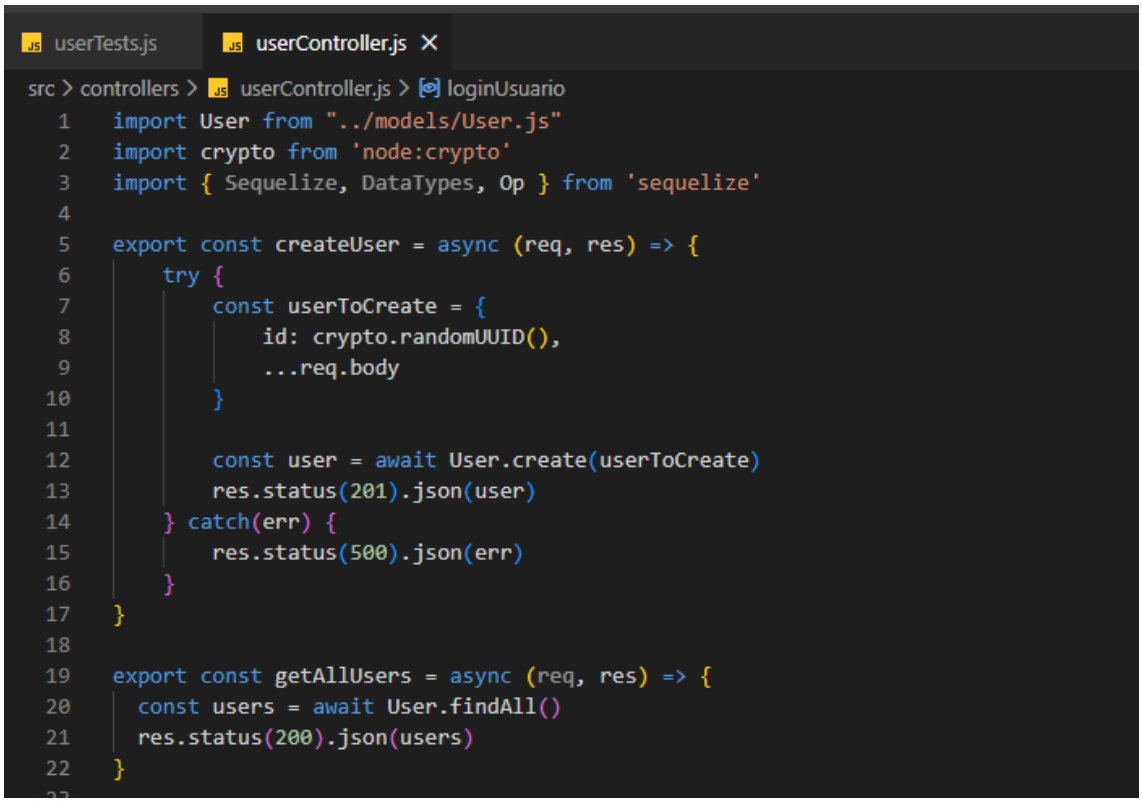


O João conseguiu. Como ele atualiza o github, não deveria dar problema.

Testes Unitário automatizado:

Users:

userController.js



```
userTests.js   userController.js
src > controllers > userController.js > loginUsuario
1   import User from "../models/User.js"
2   import crypto from 'node:crypto'
3   import { Sequelize, DataTypes, Op } from 'sequelize'
4
5   export const createUser = async (req, res) => {
6       try {
7           const userToCreate = {
8               id: crypto.randomUUID(),
9               ...req.body
10          }
11
12          const user = await User.create(userToCreate)
13          res.status(201).json(user)
14      } catch(err) {
15          res.status(500).json(err)
16      }
17  }
18
19  export const getAllUsers = async (req, res) => {
20    const users = await User.findAll()
21    res.status(200).json(users)
22  }
```

```
24  export const deleteUser = async (req, res) => {
25      const user = await User.destroy({
26          where: {id: req.params.id}
27      })
28      res.status(200).json(user)
29  }
30
31  export const getUserByID = async (req, res) => {
32    if (req.params.id === 'search') {
33      searchByPalavraChave(req, res);
34    } if (req.params.id === 'loginUsuario') {
35      loginUsuario(req, res);
36    } else {
37      const user = await User.findByPk(req.params.id);
38      res.status(200).json(user);
39    }
40  }
```

```
export const updateUser = async (req, res) => {
  const [affectedRows] = await User.update(
    { name: req.body.name,
      ...req.body },
    {
        where: {
            id: req.params.id,
        },
    }
  );
  const user = await User.findByPk(req.params.id)
  res.status(200).json(user)
}

export const searchByPalavraChave = async (req, res) => {
  const users = await User.findAll({
    where: { name: {
      [Op.like]: "%" + req.body.text + "%"
    } }
  });
  res.status(200).json(users)
}
```

```
export const loginUsuario = async (req, res) => {
  const users = await User.findAll({
    where: { username: req.body.username, password: req.body.password }
  });
  if (users.length === 1)
    res.status(200).json(users[0])
  else
    res.status(500).json("Usuário não encontrado !");
}
```

userTests.js

userTests.js

src > tests > userTests.js > ...

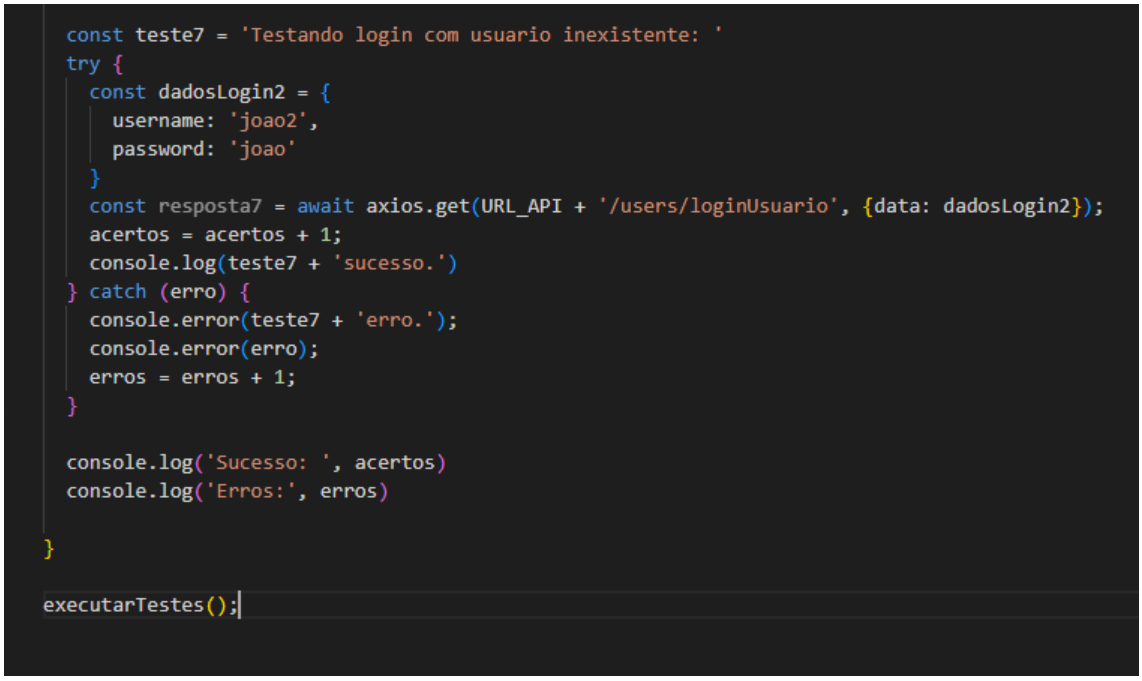
```
import axios from 'axios';

const URL_API = 'http://0.0.0.0:3000';

let erros = 0;
let acertos = 0;

async function executarTestes() {

  const teste1 = 'Testando requisicao listar tudo: '
  let userTest = '';
  try {
    const resposta1 = await axios.get(URL_API + '/users');
    userTest = resposta1.data[0].id;
    acertos = acertos + 1;
    console.log(teste1 + 'sucesso.')
  } catch (erro) {
    console.error(teste1 + 'erro.');
    console.error(erro.message);
    erros = erros + 1;
  }

  const teste2 = 'Testando requisicao listar usuario: '
  try {
    const resposta2 = await axios.get(URL_API + '/users/' + userTest);
    acertos = acertos + 1;
    console.log(teste2 + 'sucesso.')
  } catch (erro) {
    console.error(teste2 + 'erro.');
    console.error(erro.message);
    erros = erros + 1;
  }

  const teste3 = 'Testando requisicao inserir usuario: '
  let testeUsuarioID = ''
  try {
    const userData = {
      name: 'fulano de tal',
      data_nascimento: '04.04.1989',
      email: 'fultal@email.com',
      username: 'fulano',
      password: '3456as'
    }
    const resposta3 = await axios.post(URL_API + '/users/', userData);
    acertos = acertos + 1;
    console.log(teste3 + 'sucesso.')
    testeUsuarioID = resposta3.data.id
  } catch (erro) {
    console.error(teste3 + 'erro.');
    console.error(erro.message);
    erros = erros + 1;
```

```
    erros = erros + 1;
  }

  const teste4 = 'Testando requisicao atualizar usuario: '
  let testeUserID = '7758d134-0705-4928-b801-6f9fd3d2538c';

  try {
    const userDataUpdate = {
      name: 'Azevedo',
      data_nascimento: '04.03.1989',
      email: 'azevedo24@gmail.com',
      username: 'azev24',
      password: '54321as'
    }

    const resposta4 = await axios.put(URL_API + '/users/' + testeUserID, userDataUpdate);
    acertos = acertos + 1;
    console.log(teste4 + 'sucesso.')
  } catch (erro) {
    console.error(teste4 + 'erro.');
    console.error(erro.message);
    erros = erros + 1;
  }
```

userTests.js

src > tests > userTests.js > ...

```
8   async function executarTestes() {
75    const teste5 = 'Testando requisicao deletar usuario: '
76    try {
77      const resposta5 = await axios.delete(URL_API + '/users/' + testeUsuarioID);
78      acertos = acertos + 1;
79      console.log(teste5 + 'sucesso.')
80    } catch (erro) {
81      console.error(teste5 + 'erro.');
82      console.error(erro.message);
83      erros = erros + 1;
84    }
85
86    const teste6 = 'Testando login com usuario existente: '
87    try {
88      const dadosLogin = {
89        username: 'xyzpto',
90        password: '1234as'
91      }
92      const resposta6 = await axios.get(URL_API + '/users/loginUsuario', {data: dadosLogin});
93      acertos = acertos + 1;
94      console.log(teste6 + 'sucesso.')
95    } catch (erro) {
96      console.error(teste6 + 'erro.');
97      console.error(erro);
98      erros = erros + 1;
99    }
100
```



```
  const teste7 = 'Testando login com usuario inexistente: '
  try {
    const dadosLogin2 = {
      username: 'joao2',
      password: 'joao'
    }
    const resposta7 = await axios.get(URL_API + '/users/loginUsuario', {data: dadosLogin2});
    acertos = acertos + 1;
    console.log(teste7 + 'sucesso.')
  } catch (erro) {
    console.error(teste7 + 'erro.');
    console.error(erro);
    erros = erros + 1;
  }

  console.log('Sucesso: ', acertos)
  console.log('Erros:', erros)

}

executarTestes();
```

C:\Users\Latitude 3490\OneDrive\Documentos\A FIAP\A FASE 2\Avisala\avisala-challenger2> node src/tests/userTests.js

Testando requisicao listar tudo: sucesso.
Testando requisicao listar usuario: sucesso.
Testando requisicao inserir usuario: sucesso.
Testando requisicao atualizar usuario: sucesso.
Testando requisicao deletar usuario: sucesso.
Testando login com usuario existente: sucesso.

postController.js

```
export const updatePost = async (req, res) => {
  try{
  const [affectedRows] = await Post.update(
    { category: req.body.category,
      ...req.body },
    {
        where: {
            id: req.params.id,
        },
    }
  );
  const post = await Post.findByPk(req.params.id)
  res.status(200).json(post)
  }
  catch (err) {
    res.status(500).json(err)
  }
}
```

```
export const searchDescriptionByKeyword = async (req, res) => {
  const posts = await Post.findAll({
    where: {
      [Op.or]: {
        description: {[Op.like]: "%" + req.body.description + "%"},
        topic: {[Op.like]: "%" + req.body.topic + "%"}
      }
    }
  });
  res.status(200).json(posts)
}

export const searchByCategory = async (req, res) => {
  try{
  const posts = await Post.findAll({
    where: {
      category: {
                  [Op.eq]: req.params.category
                }
    }

  });
  res.status(200).json(posts);
  }
  catch (err) {
    res.status(500).json(err)
  }
}
```

postTests.js

postTests.js

src > tests > postTests.js > executarTestesPost

```
import axios from 'axios';

const URL_API = 'http://0.0.0.0:3000';

let erros = 0;
let acertos = 0;

async function executarTestesPost() {

  const teste1 = 'Testando requisicao listar todos os posts: '
  let postTest = '';
  try {
    const resposta1 = await axios.get(URL_API + '/posts');
    postTest = resposta1.data[0].id;
    acertos = acertos + 1;
    console.log(teste1 + 'sucesso.')
  } catch (erro) {
    console.error(teste1 + 'erro.');
    console.error(erro.message);
    erros = erros + 1;
  }
```

```
  const teste2 = 'Testando requisicao listar post: '
  try {
    const resposta2 = await axios.get(URL_API + '/posts/' + postTest);
    acertos = acertos + 1;
    console.log(teste2 + 'sucesso.')
  } catch (erro) {
    console.error(teste2 + 'erro.');
    console.error(erro.message);
    erros = erros + 1;
  }
```

```
const teste3 = 'Testando requisicao criar post '
let testePostID = ''
try {
  const postData = {
    category: 'História',
    topic: 'como destruir um País com apenas 1',
    description: 'Donec tempus tempus justo, non commodo sapien tempus vitae. Duis scelerisque purus',
    user_id: '7758d134-0705-4928-b801-6f9fd3d2538c'
  }
  const resposta3 = await axios.post(URL_API + '/posts/', postData);
  acertos = acertos + 1;
  console.log(teste3 + 'sucesso.')
  testePostID = resposta3.data.id;

} catch (erro) {
  console.error(teste3 + ': erro.');
  console.error(erro.message);
  erros = erros + 1;
}
```

```
const teste4 = 'Testando requisicao atualizar post: '

try {
  const postDataUpdate = {
    category: 'História',
    topic: 'Como tornar um País Forte',
    description: 'Donec tempus tempus justo, non commodo sapien tempus vitae. Duis scelerisque purus at
    userid: '7758d134-0705-4928-b801-6f9fd3d2538c'
  }

  const resposta4 = await axios.put(URL_API + '/posts/' + testePostID, postDataUpdate);
  acertos = acertos + 1;
  console.log(teste4 + 'sucesso.')
} catch (erro) {
  console.error(teste4 + 'erro.');
  console.error(erro.message);
  erros = erros + 1;
}
```

```
const teste6 = 'Testando requisicao buscar post por categoria: '
try {
  const dadosCategoria = {
    category: 'História'
  };
  const resposta6 = await axios.get(URL_API + '/posts/', {data: dadosCategoria});
  acertos = acertos + 1;
  console.log(teste6 + 'sucesso.')
} catch (erro) {
  console.error(teste6 + 'erro.');
  console.error(erro);
  erros = erros + 1;
}
```

```
const teste5 = 'Testando requisicao deletar post: '
try {
  const resposta5 = await axios.delete(URL_API + '/posts/' + testePostID);
  acertos = acertos + 1;
  console.log(teste5 + 'sucesso.')
} catch (erro) {
  console.error(teste5 + 'erro.');
  console.error(erro.message);
  erros = erros + 1;
}

console.log('Sucesso: ', acertos)
console.log('Erros:', erros)

}

executarTestesPost();
```

PS C:\Users\Latitude 3490\OneDrive\Documentos\A FIAP\A FASE 2\Avisala\avisala-challenger2> node src/tests/postTests.js

Testando requisicao listar todos os posts: sucesso.
Testando requisicao listar post: sucesso.
Testando requisicao criar post sucesso.
Testando requisicao atualizar post: sucesso.
Testando requisicao buscar post por categoria: sucesso.
Testando requisicao deletar post: sucesso.

Sucesso: 6
Erros: 0

Observação:

Caso receba o código fonte dos arquivos da estrutura um pouco diferentes é porque é muito difícil atualizar a todos. Esses *printes* de tela estão locais a minha máquina (Humberto).